



Guía Complementaria Ayudantía 1

Variables/tipos, operadores y condicionales

10 de agosto de 2026

Ayudante: Sergio Cárcamo N. - sergio.carcamon@usm.cl
Departamento de Informática UTFSM

1. Resumen de la materia (hasta ahora)

1.1. Comentarios en el código

Los comentarios son anotaciones que dejamos en nuestro código para explicar qué hace cierta parte del programa. El computador ignora estos textos por completo al momento de ejecutar el código, por lo que existen exclusivamente para ayudarnos a nosotros (o a quien revisa) a entender qué estábamos pensando al programar.

En Python, los comentarios de una sola línea se escriben anteponiendo el símbolo de “gato” o *hash* (#).

```
# Este es un comentario, el programa lo ignorará por completo  
temperatura = 45 # También podemos escribir comentarios al lado del código
```

1.2. Variables y tipos de datos

Las variables son como “cajas” o contenedores en la memoria de nuestro computador donde podemos guardar información. Cada variable tiene un nombre único que nos permite identificarla y acceder a su valor o modificarlo más adelante en la ejecución del programa.

Los cinco tipos de datos que usaremos principalmente son los siguientes:

1. **Enteros** (int): Son todos los números que van desde el $-\infty$ hasta el ∞ **sin contar decimales**. Por ejemplo: 42, 67, -341.
2. **Flotantes** (float): Representan a los números reales, es decir, **aquellos que tienen decimales**. En Python, los decimales se separan con un punto (.) y no con una coma. Por ejemplo: 3.14, -0.5, 2.0.
3. **Cadenas de texto** (str): Conocidas como *strings*, se utilizan para almacenar texto (letras, palabras, oraciones o símbolos). Siempre deben estar encerradas entre comillas simples ('Hola') o dobles ("Hola").
4. **Booleanos** (bool): Son variables lógicas que solo pueden tomar dos valores: `True` (Verdadero) o `False` (Falso). Son fundamentales para tomar decisiones lógicas en nuestro código.
5. **Listas** (list): Son estructuras que nos permiten agrupar múltiples datos (incluso de distinto tipo) de forma ordenada en una sola variable. Se definen utilizando corchetes []. Por ejemplo: [1, 2, 3] o ['manzana', 'pera'].

1.3. Declaración y redefinición de variables

Para **declarar** (crear) una variable, simplemente inventamos un nombre representativo y le asignamos un valor usando el signo igual (=).

Ojo! En programación, el signo = significa **asignación**, no igualdad matemática. El nombre de la variable siempre va a la izquierda y el valor que queremos guardar a la derecha.

Es de vital importancia entender que **las variables pueden cambiar de valor** a medida que el programa avanza. A esto le llamamos **redefinir** o actualizar una variable. Podemos sobrescribir el valor con uno nuevo, e incluso usar el valor antiguo para calcular el nuevo:

```
# Declaración inicial
juegos_steam = 4
saldo_billetera = 15000

# Redefinición directa
# Reemplazamos el valor viejo con uno completamente nuevo
saldo_billetera = 10500

# Redefinición usando su propio valor (Actualización)
# Si compramos un juego más, tomamos el valor actual (4), le sumamos 1,
# y guardamos el nuevo resultado (5) en la misma variable.
juegos_steam = juegos_steam + 1
```

1.4. Operadores Matemáticos

Para hacer cálculos en Python, utilizamos símbolos especiales llamados operadores. Además de los clásicos ya conocidos, la programación introduce un par de operadores nuevos que serán increíblemente útiles durante el semestre:

- **Suma (+) y Resta (-):** Funcionan de manera tradicional.
- **Multiplicación (*):** Se utiliza el asterisco en lugar de una "x".
- **División normal (/):** Realiza la división matemática exacta. **Siempre retorna un float (decimal)**, incluso si la división es perfecta. Por ejemplo, `10 / 2` resulta en `5.0`.
- **División entera (//):** Realiza la división pero **descarta completamente los decimales**, entregando solo la parte entera. Por ejemplo, `5 // 2` resulta en `2`. Ojo, no redondea, simplemente corta lo que está después del punto!
- **Módulo o Resto (%):** Este operador entrega el **sobranante** de una división entera. Por ejemplo, si tenemos 5 manzanas y las dividimos entre 2 personas, cada uno recibe 2 y sobra 1. Por lo tanto, `5 % 2` resulta en `1`. Es fundamental, por ejemplo, para saber si un número es par o impar.
- **Exponenciación (**):** Sirve para elevar una base a un exponente. Por ejemplo, `2 ** 3` equivale a 2^3 , dando como resultado 8.

```
# Ejemplos de uso
saldo = 15000
precio_juego = 4000

# Cuantos juegos puedo comprar sin quedar debiendo? (División entera)
juegos_comprables = saldo // precio_juego # Resultado: 3

# Cuánto dinero me sobraría tras comprar esos 3 juegos? (Módulo)
vuelto = saldo % precio_juego # Resultado: 3000

# Calculando componentes
memoria_ram = 2 ** 4 # Resultado: 16
```

Orden de precedencia y uso de paréntesis

Al igual que en las matemáticas tradicionales, Python respeta un orden de operaciones estricto. Primero se resuelven los paréntesis, luego las potencias, seguido de las multiplicaciones y divisiones (incluyendo división entera y módulo) de izquierda a derecha, y finalmente las sumas y restas.

Si necesitas alterar este orden o simplemente quieres que tu cálculo sea más seguro y fácil de leer, **debes usar paréntesis curvos ()**. Ojo! En Python matemático no se usan corchetes [] ni llaves {} para agrupar operaciones, solo paréntesis curvos, sin importar cuántos necesites anidar.

```
# Ejemplo de precedencia
# Primero multiplica (5*2), luego suma 10. Resultado: 20
resultado1 = 10 + 5 * 2

# Primero resuelve el paréntesis (10+5), luego multiplica. Resultado: 30
resultado2 = (10 + 5) * 2

# Uso de multiples paréntesis para asegurar el orden
calculo_complejo = ((3 + 4) * 2) ** 2
```

1.5. Entrada y salida

Para que nuestro programa sea interactivo, necesita comunicarse con el usuario. Para esto utilizamos dos funciones fundamentales:

- **Salida (print())**: Muestra información en la pantalla. Podemos imprimir texto, el valor de una variable, o varias cosas a la vez separándolas por comas.
- **Entrada (input())**: Pausa el programa y espera a que el usuario escriba algo por el teclado y presione *Enter*. Lo que el usuario escriba será guardado en una variable.

```
# Salida de datos
nombre = "Karina"
edad = 21
# Imprimiendo texto y variables juntas
print("Hola", nombre, "tienes", edad, "años.")

# Entrada de datos
# El programa se detiene a esperar que el usuario escriba
comida = input("Ingresa tu comida favorita: ")
print("Tu comida favorita es:", comida)
```

1.6. Casteo (conversión)

El **casteo** (o conversión) es la acción de transformar un tipo de dato en otro compatible utilizando las funciones nativas de Python: `int()`, `float()`, `str()` y `bool()`.

En programación, a menudo nos encontraremos con situaciones donde un dato no está en el formato que necesitamos. Por ejemplo, intentar realizar cálculos matemáticos con texto, o querer unir un número a una oración sin que Python nos arroje un error. Aquí es donde entra en juego la conversión explícita.

A continuación, detallamos las conversiones más importantes y cómo usarlas a tu favor:

- **De float a int:** Al convertir un decimal a entero, Python simplemente "corta" la parte decimal, no la redondea. Por ejemplo, `int(3.9)` dará como resultado 3. Hay que tener ojo, porque esto implica una pérdida de información.
- **De int a float:** Se añade un punto decimal vacío. Por ejemplo, `float(5)` resulta en 5.0.
- **De str a numérico:** Solo funciona si el texto tiene estrictamente un formato numérico. Por ejemplo, `int("10")` es válido, pero `int("Hola")` o intentar pasar decimales directos como `int("10.5")` provocará que tu programa colapse (error).
- **De numérico a str:** Nos permite transformar cualquier número en texto. Esto es sumamente útil cuando queremos concatenar (unir) números con otras cadenas de texto de forma precisa.
- **Casos especiales (Booleanos):** En Python, matemáticamente `True` equivale al número 1, y `False` equivale al número 0.

El peligro de `input()` y la trampa del texto

El casteo es particularmente crítico cuando solicitamos datos con la función `input()`. **Debes recordar siempre que `input()` retorna una variable de tipo `str` (texto)**, incluso si el usuario teclea un número por su teclado.

Si intentamos sumar dos entradas de texto sin castearlas previamente, Python no las sumará matemáticamente, sino que las **concatenará** (las unirá como si fueran palabras).

```
# El problema de no castear
# Supongamos que el usuario ingresa 5000 y luego 2500
saldo_steam = input("Ingrese su saldo de Steam: ")
tarjeta_regalo = input("Ingrese el monto de la tarjeta de regalo: ")

# Esto no dará 7500. Como son textos, Python los pega y el resultado será
# "50002500"
saldo_total_erroneo = saldo_steam + tarjeta_regalo

# La solución correcta
# Casteamos el input a int inmediatamente al recibirlo
saldo_steam = int(input("Ingrese su saldo de Steam: "))
tarjeta_regalo = int(input("Ingrese el monto de la tarjeta de regalo: "))

# Ahora si, Python reconoce que son números y realizará una suma matemática
saldo_total_correcto = saldo_steam + tarjeta_regalo
```

Pérdida de información al castear flotantes

Otra situación muy común es cuando necesitamos forzar un número decimal a ser un entero. Es vital recordar que `int()` simplemente trunca el número, destruyendo la precisión de los decimales.

```
# Supongamos que monitoreamos la temperatura de una tarjeta de video (GPU)
temp_gpu_exacta = 78.9

# Si casteamos a entero, perdemos el .9 por completo
temp_gpu_entera = int(temp_gpu_exacta)

print("Temperatura exacta:", temp_gpu_exacta) # Imprime 78.9
print("Temperatura truncada:", temp_gpu_entera) # Imprime 78 (Perdimos casi un grado!)
```

1.7. Librerías y funciones útiles

Por ahora, una de las funciones nativas más útiles que veremos es `round()`. A diferencia del casteo a entero que simplemente trunca los decimales, `round(numero, decimales)` redondea de manera matemática un número a la cantidad de decimales que le indiquemos. Si no le entregamos la cantidad de decimales, lo redondeará al entero más cercano.

```
pi = 3.14159
# Redondear a 2 decimales
pi_redondeado = round(pi, 2)
print(pi_redondeado) # Imprimirá 3.14
```

Sin embargo, Python tiene muchísimas herramientas matemáticas más avanzadas (como la raíz cuadrada o la trigonometría) guardadas en una “caja de herramientas” externa llamada **librería** (o módulo).

Para usar estas herramientas matemáticas, primero debemos decirle a nuestro programa que abra esa caja usando la instrucción `import math`. Esto idealmente debe ir en la primera línea de tu archivo de código.

En nuestro caso, utilizaremos más que nada `sqrt(x)` para calcular la raíz cuadrada exacta de un valor `x`. De esta forma, podemos importar únicamente esa función usando directamente la instrucción `from math import sqrt`.

```
from math import sqrt

# Ejemplo: Distancia en un mapa digital (Pitágoras)
cateto_a = 3
cateto_b = 4

# Aplicamos el teorema: c = raíz_cuadrada(a^2 + b^2)
suma_cuadrados = (cateto_a ** 2) + (cateto_b ** 2)
hipotenusa = sqrt(suma_cuadrados)

print("La distancia es:", hipotenusa) # Resultado: 5.0
```

1.8. Decisiones (Control de flujo)

Hasta ahora, nuestros programas se ejecutan línea por línea, de arriba hacia abajo, sin saltarse nada. Sin embargo, en la programación muchas veces necesitamos tomar **decisiones**. Queremos que una parte del código se ejecute *solo si* se cumple una condición específica.

Para esto utilizamos la estructura `if` (si condicional), la cual sigue algunas reglas estructurales muy importantes:

- **El `if` es obligatorio e inicial:** Toda decisión arranca con un `if`. Además, un `if` puede vivir completamente solo si no necesitas revisar otros escenarios.
- **Puedes tener muchos `elif` (opcionales):** Si quieres evaluar múltiples condiciones distintas en cadena una tras otra, puedes agregar la cantidad de `elif` que necesites.
- **Solo un `else` (opcional):** Si quieres atrapar cualquier caso que no haya cumplido **ninguna** de las condiciones anteriores, usas el `else`. Solo puede haber **uno** por estructura y debe ir siempre al final.

Para evaluar estas condiciones, usamos **operadores relacionales** que comparan dos valores y siempre entregan un booleano (`True` o `False`):

- `==` : Igual a (Ojo! Son dos signos igual, recuerda que uno solo es para variables).
- `!=` : Distinto de.
- `>` y `<` : Mayor que, y Menor que.
- `>=` y `<=` : Mayor o igual que, y Menor o igual que.

Regla de oro de Python: El bloque de código que está "dentro" de una decisión se define usando **indentación** (una sangría o tabulación hacia la derecha). Todo lo que esté indentado bajo el `if` se ejecutará solo si la condición es verdadera.

```
# Ejemplo 1: if y else básico
saldo_billetera = 15000
precio_juego = 12000

# Verificamos si el saldo alcanza para el juego
if saldo_billetera >= precio_juego:
    print("Compra exitosa. Agregado a tu biblioteca de juegos.")
    saldo_billetera = saldo_billetera - precio_juego
    print("Tu nuevo saldo es:", saldo_billetera)

# Si la condición de arriba fue False, el programa salta directo aquí
else:
    dinero_faltante = precio_juego - saldo_billetera
    print("Saldo insuficiente. Faltan", dinero_faltante, "pesos.")

# Ejemplo 2: Múltiples decisiones con elif
temp_cpu = 82

# Solo necesitamos un if para empezar
if temp_cpu >= 90:
    print("Peligro! Temperatura critica. Apagando equipo...")

# Podemos tener tantos elif como queramos.
# Este se revisa SÓLO si el 'if' anterior fue falso
elif temp_cpu >= 75:
    print("Advertencia: Procesador caliente. Revisa el ventilador y la pasta térmica.")

# Solo puede haber un else, y va al final atrapando todo lo que sobró
else:
    print("Sistema estable. Temperatura optima.")
```

1.9. Operadores Lógicos

Muchas veces una sola condición no es suficiente para tomar una decisión. Qué pasa si necesitamos que se cumplan *dos cosas al mismo tiempo*, o que se cumpla *por lo menos una de varias*? Para esto utilizamos los operadores lógicos, los cuales nos permiten combinar o modificar múltiples condiciones:

- **and** (Y): Exige que **todas** las condiciones conectadas sean Verdaderas (True). Si al menos una es falsa, todo el bloque será evaluado como falso.
- **or** (O): Solo necesita que **al menos una** de las condiciones sea Verdadera. Será falso únicamente si todas las condiciones son absolutamente falsas.
- **not** (Negación): Invierte el valor lógico de una condición. Lo verdadero lo vuelve falso, y lo falso lo vuelve verdadero.

Al igual que con las matemáticas, es una excelente práctica usar **paréntesis** para separar tus condiciones lógicas. Así evitas confusiones sobre qué se está evaluando primero.

```
edad = 20
tiene_entrada = True
dia_semana = "Viernes"

# Uso de AND: Se deben cumplir AMBAS condiciones
if (edad >= 18) and (tiene_entrada == True):
    print("Puedes ingresar al concierto.")
else:
    print("No cumples los requisitos para entrar.")

# Uso de OR: Basta con que se cumpla UNA condición
if (dia_semana == "Sabado") or (dia_semana == "Domingo"):
    print("Es fin de semana, a descansar!")
else:
    print("Aún es día de semana.")

# Uso de NOT: Invierte la condición
juego_terminado = False

if not juego_terminado:
    print("Sigue jugando, la partida aún no termina!")
```

2. Ejercicios Propuestos

Ejercicio 1: El cajero automático básico

Usted ha sido contratado para programar la lógica básica de un cajero automático. El cajero solo entrega billetes de \$10.000. Escriba un programa que le pregunte al usuario cuánto dinero total tiene en su cuenta y luego le pregunte cuánto dinero desea retirar. El programa debe calcular y mostrar:

1. Cuántos billetes de \$10.000 se le entregarán al usuario.
2. Cuánto dinero quedará exactamente en su cuenta después del retiro.

Nota: Asuma que el usuario siempre pide un monto que es múltiplo de 10.000 y que tiene saldo suficiente.

Ejercicio 2: Conversión de Tiempo

En astronomía, a veces los tiempos de los eventos se calculan en una cantidad gigante de segundos. Escriba un programa que reciba una cantidad de segundos como número entero e imprima a cuántas horas, minutos y segundos equivale.

Ejemplo: Si el usuario ingresa 3665, el programa debe imprimir 1 horas, 1 minutos y 5 segundos.

Ejercicio 3: El promedio ponderado

En la universidad, las notas finales de un ramo se calculan con promedios ponderados. Suponga que un ramo tiene tres evaluaciones con las siguientes ponderaciones:

- Certamen 1: 30 %
- Certamen 2: 30 %
- Proyecto Final: 40 %

Escriba un programa que le pida al estudiante sus tres notas (que pueden tener decimales) y calcule su nota final del semestre.

Ejercicio 4: El encriptador numérico

Para enviar mensajes secretos, un espía convierte cada letra en un número, formando un código largo. Para evitar ser descubierto, el espía necesita un programa que aplique un hash matemático simple a un número entero de exactamente **cuatro dígitos** (por ejemplo, 4183).

El algoritmo de encriptación que debes programar sigue estos pasos:

1. Extraer el primer dígito (el de los miles) y el último dígito (el de las unidades).
2. Sumar ambos dígitos.
3. Multiplicar esa suma por el número original completo.
4. El resultado final es el código encriptado a imprimir.

Restricción obligatoria: Está estrictamente prohibido usar `str()` para tratar el número como texto y extraer sus caracteres. Debe resolverse usando únicamente divisiones enteras (`//`) y módulo (`%`).

Ejercicio 5: Calculadora de Envíos de Hardware

Estás comprando componentes electrónicos y el costo de envío depende del peso total del paquete (en gramos) y la distancia (en kilómetros). La fórmula del costo base es:

$$\text{Costo} = (\text{peso} \cdot 5) + (\text{distancia} \cdot 10)$$

Además, se aplican los siguientes recargos o descuentos según el costo base calculado:

- Si el costo base es estrictamente menor a \$5.000, se cobra un recargo fijo de \$1.000 por embalaje.
- Si el costo base está entre \$5.000 y \$10.000 (ambos inclusive), no hay recargos extra.
- Si el costo base es estrictamente mayor a \$10.000 y hasta \$20.000 (inclusive), se aplica un descuento del 10 % al costo base.
- Si el costo base supera los \$20.000, el envío tiene un súper descuento del 20 %.

Escriba un programa que solicite al usuario el peso y la distancia, calcule el costo final de envío y lo muestre en pantalla redondeado a cero decimales.

Ejercicio 6: El Kiosco Universitario

En el recreo, los estudiantes arrasan con el kiosco. Escriba un programa que funcione como la caja registradora básica del kiosco. Debe pedirle al usuario que ingrese la cantidad de **Chaparritas** (a \$3.000 cada una) y la cantidad de **Galletas** (a \$800 cada una) que el cliente desea comprar.

El programa debe calcular el total a pagar y mostrarlo por pantalla.

Ejercicio 7: Ofertas en la Tienda de Juegos

Estamos en época de ofertas de verano en Steam y necesitas saber exactamente cuánto te costará un juego en dólares después del descuento. Escriba un programa que solicite el precio original del juego (puede tener decimales, por ejemplo, 29,99) y el porcentaje de descuento (un número entero, por ejemplo, 75 para un 75 % de descuento).

El programa debe calcular el precio final a pagar y mostrarlo redondeado a **dos decimales**.

Pista: Para calcular cuánto se descuenta, multiplica el precio por el porcentaje y divídelo en 100.

Ejercicio 8: El Multiplicador de Daño Crítico

En un juego de rol (RPG), el daño que hace un personaje a un enemigo depende de su nivel de ataque, la defensa del enemigo y de un multiplicador que varía según la diferencia de niveles. El daño base se calcula como:

$$D = \text{Ataque} \cdot (\text{Ataque} / \text{Defensa})$$

Reglas del multiplicador de nivel (Diferencia = Nivel Jugador - Nivel Enemigo):

- Si la diferencia es menor o igual a -5 (el enemigo te supera por 5 o más niveles): el multiplicador es 0.5.
- Si la diferencia es menor a 0 (el enemigo tiene mayor nivel, pero por menos de 5): el multiplicador es 0.8.
- Si tienen exactamente el mismo nivel (diferencia 0): el multiplicador es 1.0.

- Si la diferencia es mayor a 0 y menor a 5 (eres mayor nivel por poco): el multiplicador es 1.2.
- Si la diferencia es 5 o mayor (eres muy superior): daño crítico, el multiplicador es 1.5.

Escriba un programa que solicite el ataque y nivel del jugador, además de la defensa y nivel del enemigo. Calcule el daño final (daño base · multiplicador) y muéstrelo redondeado a un decimal.

Ejercicio 9: Seguridad y Refrigeración del PC

Estás programando un sistema de seguridad interno para un computador de escritorio. El sistema debe decidir si permite que el equipo siga operando con normalidad o si debe forzar un apagado de emergencia para proteger los componentes.

Escriba un programa que solicite:

1. La temperatura actual del Procesador (CPU).
2. La temperatura actual de la Tarjeta de Video (GPU).
3. Las RPM (Revoluciones Por Minuto) del ventilador principal.

Utilizando operadores lógicos (and, or), el sistema debe imprimir: "Apagado de emergencia activado!" si ocurre **alguna** de las siguientes situaciones:

- La temperatura de la CPU supera los 90 grados **O** la temperatura de la GPU supera los 95 grados.
- **Ambas** temperaturas (CPU y GPU) son mayores a 80 grados, **Y además** el ventilador está girando a menos de 1000 RPM (indicando un fallo crítico de refrigeración).

Si no se cumple ninguna de estas condiciones de riesgo, el programa debe imprimir: "Sistema estable. Temperaturas óptimas.".

Ejercicio 10: Conectando la Placa de Circuitos (PCB)

Estás diseñando una placa de circuitos para un proyecto con microcontroladores y necesitas conectar dos pines con un cable (*jumper*). Conoces las coordenadas (x_1, y_1) y (x_2, y_2) de ambos pines en un plano (medidas en milímetros).

Escriba un programa que solicite al usuario las cuatro coordenadas (como números flotantes). Luego, utilizando la fórmula matemática de distancia entre dos puntos, calcule y muestre la longitud exacta del cable que se necesita para conectarlos en línea recta.

El resultado final debe mostrarse **redondeado a 2 decimales**.

Recordatorio de la fórmula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Ejercicio 11: El Sistema de Riego Automático

Un aspersor de agua riega el pasto girando en 360 grados, formando un círculo perfecto. Escriba un programa que le pida al usuario el radio de alcance del aspersor (en metros, puede tener decimales).

El programa debe calcular e imprimir:

1. El perímetro del círculo de riego (Fórmula: $2 \cdot \pi \cdot r$).
2. El área total del pasto que se está regando (Fórmula: $\pi \cdot r^2$).

Ambos resultados deben utilizar el valor de Pi aproximado a 3.14159 y deben mostrarse redondeados a **un solo decimal**.

Ejercicio 12: Pintando la Habitación

Vas a pintar las cuatro paredes de tu habitación y necesitas saber cuántos galones de pintura comprar. Escriba un programa que solicite el largo, el ancho y la altura de la habitación (en metros). Considere que el techo y el piso **no** se pintan, y para simplificar, asuma que la habitación no tiene puertas ni ventanas.

El programa debe calcular el área total de las cuatro paredes. Luego, sabiendo que 1 galón de pintura rinde exactamente para **12 metros cuadrados**, calcule cuántos galones exactos se necesitan y muestre el resultado final redondeado a **dos decimales**.

3. Soluciones Propuestas

Notar que las soluciones propuestas de cada ejercicio **no necesariamente** son las únicas o mejores. De la misma forma, en evaluaciones (certámenes, tareas) se evaluará la funcionalidad y el pensamiento lógico por sobre la implementación, por lo que se recomienda comentar para que el evaluador sepa qué es lo que se quiso decir.

Solución Ejercicio 1

Solución Esperada

Para calcular la cantidad de billetes, aprovechamos la división entera (`//`), la cual nos dice exactamente cuántas veces cabe el valor de 10.000 en el monto solicitado. Para obtener el saldo final, realizamos una simple resta.

```
saldo_inicial = int(input("Ingrese su saldo total: "))
monto_retiro = int(input("Ingrese monto a retirar: "))

# Calculamos los billetes usando division entera
billetes_10k = monto_retiro // 10000

# Calculamos el nuevo saldo
saldo_final = saldo_inicial - monto_retiro

print("Se entregarán", billetes_10k, "billetes de $10.000")
print("Su nuevo saldo es:", saldo_final)
```

Solución Ejercicio 2

Solución Esperada

Este es un ejercicio clásico de conversión. La división entera (`//`) nos da la unidad mayor (cuántas horas o minutos completos hay), mientras que el módulo (`%`) rescata el "sobran-te"(los segundos restantes) para seguir subdividiéndolos.

```
total_segundos = int(input("Ingrese la cantidad de segundos: "))

# Una hora tiene 3600 segundos (60 * 60)
horas = total_segundos // 3600

# Nos quedamos con los segundos sobrantes despues de sacar las horas
segundos_restantes = total_segundos % 3600

# De esos sobrantes, sacamos los minutos
minutos = segundos_restantes // 60

# Lo que sobra finalmente son los segundos
segundos_finales = segundos_restantes % 60

print(horas, "horas,", minutos, "minutos y", segundos_finales, "segundos.")
```

Solución Ejercicio 3

Solución Esperada

Aquí transformamos los porcentajes a su equivalente decimal (ej: 30% pasa a ser 0.3) para calcular la ponderación. Usamos `float()` en la entrada de datos, ya que las notas universitarias suelen incluir decimales.

```
# Usamos float() porque las notas pueden tener decimales (ej: 85.5)
c1 = float(input("Ingrese nota del Certamen 1: "))
c2 = float(input("Ingrese nota del Certamen 2: "))
proyecto = float(input("Ingrese nota del Proyecto: "))

# Multiplicamos por los porcentajes (30% = 0.3)
nota_final = (c1 * 0.30) + (c2 * 0.30) + (proyecto * 0.40)

# Redondeamos a 1 decimal como es costumbre (opcional para los alumnos)
nota_final = round(nota_final, 1)

print("Tu nota final del semestre es:", nota_final)
```

Solución Ejercicio 4

Solución Esperada

Al usar `// 1000`, eliminamos los últimos tres dígitos de un número de 4 cifras, quedándonos solo con el primero. Por otro lado, al aplicar módulo base 10 (`% 10`) a cualquier número, siempre obtendremos exclusivamente su unidad.

```
numero_original = int(input("Ingrese un número de 4 dígitos: "))

# Ejemplo mental con 4183:
# Para sacar el primer dígito (4), dividimos por 1000 y botamos los
# decimales
primer_digito = numero_original // 1000

# Para sacar el último dígito (3), pedimos el resto de la división por
# 10
ultimo_digito = numero_original % 10

# Realizamos las operaciones matemáticas solicitadas
suma_extremos = primer_digito + ultimo_digito
codigo_encriptado = suma_extremos * numero_original

print("El código encriptado es:", codigo_encriptado)
```

Solución Ejercicio 5

Solución Esperada

Primero, se calcula un costo base preliminar usando la fórmula entregada. Luego, utilizamos una cascada de `if` / `elif` / `else` para evaluar en qué tramo monetario cae ese costo y aplicar el recargo o descuento. Al usar `elif`, aseguramos que el programa evalúe un tramo a la vez de forma segura.

```
peso = float(input("Ingrese el peso del paquete (g): "))
distancia = float(input("Ingrese la distancia (km): "))

# Calculamos el costo base con la formula dada
costo_base = (peso * 5) + (distancia * 10)

# Evaluamos los recargos y descuentos
if costo_base < 5000:
    costo_final = costo_base + 1000
elif costo_base <= 10000:
    costo_final = costo_base
elif costo_base <= 20000:
    # Descuento del 10% significa que pagamos el 90%
    costo_final = costo_base * 0.90
else:
    # Descuento del 20% significa que pagamos el 80%
    costo_final = costo_base * 0.80

print("El costo de envio final es: $", round(costo_final))
```

Solución Ejercicio 6

Solución Esperada

Se aplica una lógica directa de compras: multiplicamos el precio unitario de cada producto por la cantidad ingresada, para luego sumar ambos subtotales y obtener el valor total.

```
cant_chaparritas = int(input("Cuántas chaparritas desea?: "))
cant_galletas = int(input("Cuántas galletas desea?: "))

total_chaparritas = cant_chaparritas * 3000
total_galletas = cant_galletas * 800

total_a_pagar = total_chaparritas + total_galletas

print("El total a pagar es: $", total_a_pagar)
```

Solución Ejercicio 7

Solución Esperada

Calculamos el monto exacto de la rebaja aplicando una regla de tres simple (multiplicamos el precio por el porcentaje y lo dividimos en 100). Finalmente, restamos ese monto al valor original del juego.

```
precio_original = float(input("Ingrese el precio original del juego (
    USD): "))
porcentaje_descto = int(input("Ingrese el porcentaje de descuento: "))

# Calculamos el dinero que nos van a rebajar
monto_descontado = (precio_original * porcentaje_descto) / 100

# Calculamos el precio final y lo redondeamos a 2 decimales
precio_final = precio_original - monto_descontado
precio_final = round(precio_final, 2)

print("El precio final con descuento es: $", precio_final)
```

Solución Ejercicio 8

Solución Esperada

Tras calcular el daño base según la fórmula, se determina la diferencia de niveles. El bloque de validaciones está ordenado estrictamente desde el caso más negativo al positivo; el orden de los `elif` es crucial para que Python no asigne un multiplicador incorrecto antes de tiempo.

```
ataque = float(input("Ataque del jugador: "))
nivel_jugador = int(input("Nivel del jugador: "))
defensa = float(input("Defensa del enemigo: "))
nivel_enemigo = int(input("Nivel del enemigo: "))

# Calculamos el daño base
dano_base = ataque * (ataque / defensa)
diferencia = nivel_jugador - nivel_enemigo

# Determinamos el multiplicador con multiples elif
if diferencia <= -5:
    multiplicador = 0.5
elif diferencia < 0:
    multiplicador = 0.8
elif diferencia == 0:
    multiplicador = 1.0
elif diferencia < 5:
    multiplicador = 1.2
else:
    multiplicador = 1.5

dano_final = dano_base * multiplicador
print("El daño final infligido es:", round(dano_final, 1))
```

Solución Ejercicio 9

Solución Esperada

Este ejercicio es ideal para aplicar paréntesis y ordenar nuestra lógica. Podemos evaluar todo dentro de un mismo `if`, separando el primer escenario (temperaturas extremas individuales) del segundo escenario (temperaturas altas sumado a un fallo del ventilador) mediante un `or` principal.

```
temp_cpu = float(input("Ingrese temperatura de CPU: "))
temp_gpu = float(input("Ingrese temperatura de GPU: "))
rpm_ventilador = int(input("Ingrese RPM del ventilador: "))

# Agrupamos las condiciones con parentesis para asegurar que se evalúen
# en el orden correcto
if (temp_cpu > 90 or temp_gpu > 95) or (temp_cpu > 80 and temp_gpu > 80
    and rpm_ventilador < 1000):
    print("Apagado de emergencia activado!")
else:
    print("Sistema estable. Temperaturas óptimas.")
```

Solución Ejercicio 10

Solución Esperada

Calculamos las diferencias de las coordenadas y las elevamos al cuadrado de inmediato. Luego, para el último paso de Pitágoras, invocamos `math.sqrt()` desde la librería importada para resolver la raíz cuadrada.

```
from math import sqrt

# Solicitamos las 4 coordenadas
x1 = float(input("Ingrese x1: "))
y1 = float(input("Ingrese y1: "))
x2 = float(input("Ingrese x2: "))
y2 = float(input("Ingrese y2: "))

# Calculamos las diferencias y las elevamos al cuadrado
dif_x_cuadrado = (x2 - x1) ** 2
dif_y_cuadrado = (y2 - y1) ** 2

# Sumamos y aplicamos raíz cuadrada (sqrt)
distancia = sqrt(dif_x_cuadrado + dif_y_cuadrado)

# Redondeamos a 2 decimales
distancia_final = round(distancia, 2)

print("La longitud del cable debe ser de", distancia_final, "milímetros
.")
```

Solución Ejercicio 11

Solución Esperada

Dado que no utilizamos herramientas de la librería, declaramos manualmente la variable Pi al comienzo del código para emplearla repetidamente como una constante en las fórmulas clásicas de perímetro y área.

```
pi = 3.14159
radio = float(input("Ingrese el radio de alcance del aspersor (m): "))

# Calculamos perimetro y area usando la aproximacion de pi
perimetro = 2 * pi * radio
area = pi * (radio ** 2)

# Redondeamos a 1 decimal
perimetro_redondeado = round(perimetro, 1)
area_redondeada = round(area, 1)

print("El perímetro de riego es:", perimetro_redondeado, "metros.")
print("El área total regada es:", area_redondeada, "metros cuadrados.")
```

Solución Ejercicio 12

Solución Esperada

Agrupamos las paredes por pares usando una simple multiplicación base por altura. Al dividir el área total por 12, es altamente probable obtener decimales infinitos, los cuales restringimos finalmente utilizando la función round().

```
largo = float(input("Ingrese el largo de la habitacion (m): "))
ancho = float(input("Ingrese el ancho de la habitacion (m): "))
altura = float(input("Ingrese la altura de la habitacion (m): "))

# Hay dos paredes de (largo * altura) y dos paredes de (ancho * altura)
area_paredes_largas = 2 * (largo * altura)
area_paredes anchas = 2 * (ancho * altura)

area_total = area_paredes_largas + area_paredes anchas

# Dividimos en 12 para saber los galones (esto dará un decimal)
galones_exactos = area_total / 12

# Redondeamos a dos decimales
galones_comprar = round(galones_exactos, 2)

print("El área total a pintar es de", round(area_total, 2), "metros cuadrados.")
print("Debes comprar", galones_comprar, "galones de pintura.")
```